NZTP: 01

REST - Representational State Transfer

Styl architektoniczny wykorzystywany w projektowaniu internetowych usług sieciowych.

Zasoby identyfikowane przez **URI** (Uniform Resource Identifier)

Komunikaty zwykle w formie **JSON** (JavaScript Object Notation)

- GET - Pobranie zasobu
- POST - Tworzenie zasobu
- PUT - Aktualizacja zasobu
- DELETE - Usunięcie zasobu
- PATCH - Częściowa aktualizacja zasobu

URI

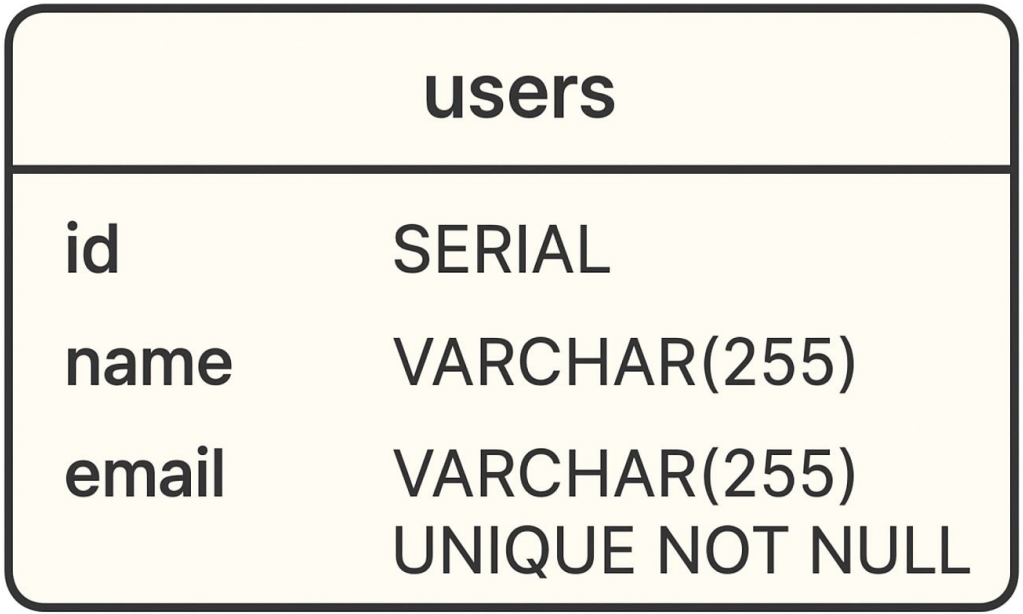
Jednolity identyfikator zasobu

scheme://authority/path?query#fragment

https://example.com/api/users/10?active=true#info

- https **scheme** Określa protokół komunikacji (np. http, https, ftp, mailto)
- example.com **authority** Zwykle domena lub adres IP serwera
- /api/users/10 **path** Określa konkretny zasób
- ?active=true **query** (zapytanie) Parametry przekazywane do serwera #info **fragment** Odnosi się do części dokumentu

REST: przykład - użytkownicy



Python - FastAPI + SQLAlchemy + Pydantic

database.py

```python
from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker

SQLALCHEMY_DATABASE_URL = "postgresql+psycopg2://postgres:password@localhost:5432/usersdb"

engine = create_engine(SQLALCHEMY_DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()
```

models.py

```python
from sqlalchemy import Column, Integer, String
from .database import Base

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String, nullable=False)
    email = Column(String, unique=True, index=True, nullable=False)
```

schemas.py

```python
from pydantic import BaseModel, EmailStr

class UserBase(BaseModel):
    name: str
    email: EmailStr

class UserCreate(UserBase):
    pass

class User(UserBase):
    id: int

    class Config:
        orm_mode = True
```

Python - FastAPI + SQLAlchemy + Pydantic

```python
from fastapi import FastAPI, Depends, HTTPException
from sqlalchemy.orm import Session
from . import models, schemas, database

models.Base.metadata.create_all(bind=database.engine)

app = FastAPI(title="FastAPI + PostgreSQL")

def get_db():
    db = database.SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Python - FastAPI + SQLAlchemy + Pydantic

main.py

```python
@app.get("/users", response_model=list[schemas.User])
def get_users(db: Session = Depends(get_db)):
    return db.query(models.User).all()

@app.get("/users/{id}", response_model=schemas.User)
def get_user(id: int, db: Session = Depends(get_db)):
    user = db.query(models.User).filter(models.User.id == id).first()
    if not user:
        raise HTTPException(404, "User not found")
    return user

@app.post("/users", response_model=schemas.User)
def create_user(user: schemas.UserCreate, db: Session = Depends(get_db)):
    db_user = models.User(**user.dict())
    db.add(db_user)
    db.commit()
    db.refresh(db_user)
    return db_user

@app.put("/users/{id}", response_model=schemas.User)
def update_user(id: int, updated: schemas.UserCreate, db: Session = Depends(get_db)):
    user = db.query(models.User).filter(models.User.id == id).first()
    if not user:
        raise HTTPException(404, "User not found")
    user.name = updated.name
    user.email = updated.email
    db.commit()
    db.refresh(user)
    return user
```

```python
@app.patch("/users/{id}", response_model=schemas.User)
def patch_user(id: int, data: dict, db: Session = Depends(get_db)):
    user = db.query(models.User).filter(models.User.id == id).first()
    if not user:
        raise HTTPException(404, "User not found")
    for key, value in data.items():
        setattr(user, key, value)
    db.commit()
    db.refresh(user)
    return user

@app.delete("/users/{id}")
def delete_user(id: int, db: Session = Depends(get_db)):
    user = db.query(models.User).filter(models.User.id == id).first()
    if not user:
        raise HTTPException(404, "User not found")
    db.delete(user)
    db.commit()
    return {"message": "User deleted"}
```

C# - ASP.NET Core + PostgreSQL (Entity Framework Core)

appsettings.json

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Host=localhost;Database=usersdb;Username=postgres;Password=password"
  }
}
```

User.cs

```
namespace UsersApi.Models;

public class User
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public string Email { get; set; } = "";
}
```

AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using UsersApi.Models;

namespace UsersApi.Data;

public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options) { }

    public DbSet<User> Users => Set<User>();
}
```

C# - ASP.NET Core + PostgreSQL (Entity Framework Core)

Program.cs

```
using Microsoft.EntityFrameworkCore;
using UsersApi.Data;
using UsersApi.Models;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<AppDbContext>(opt =>
    opt.UseNpgsql(builder.Configuration.GetConnectionString("DefaultConnection")));
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();
app.UseSwagger();
app.UseSwaggerUI();

app.MapGet("/users", async (AppDbContext db) => await db.Users.ToListAsync());

app.MapGet("/users/{id}", async (int id, AppDbContext db) =>
    await db.Users.FindAsync(id) is User user ? Results.Ok(user) : Results.NotFound());

app.MapPost("/users", async (User user, AppDbContext db) =>
{
    db.Users.Add(user);
    await db.SaveChangesAsync();
    return Results.Created($"/users/{user.Id}", user);
});

app.MapPut("/users/{id}", async (int id, User updated, AppDbContext db) =>
{
    var user = await db.Users.FindAsync(id);
    if (user is null) return Results.NotFound();
    user.Name = updated.Name;
    user.Email = updated.Email;
    await db.SaveChangesAsync();
    return Results.Ok(user);
});

app.MapPatch("/users/{id}", async (int id, Dictionary<string, string> data, AppDbContext db) =>
{
    var user = await db.Users.FindAsync(id);
    if (user is null) return Results.NotFound();
    foreach (var kv in data)
    {
        if (kv.Key == "Name") user.Name = kv.Value;
        if (kv.Key == "Email") user.Email = kv.Value;
    }
    await db.SaveChangesAsync();
    return Results.Ok(user);
});

app.MapDelete("/users/{id}", async (int id, AppDbContext db) =>
{
    var user = await db.Users.FindAsync(id);
    if (user is null) return Results.NotFound();
    db.Users.Remove(user);
    await db.SaveChangesAsync();
    return Results.Ok(new { message = "User deleted" });
});

app.Run();
```

Java - Spring Boot + PostgreSQL (Spring Data JPA)

application.properties

```
spring.datasource.url=jdbc:postgresql://localhost:5432/usersdb
spring.datasource.username=postgres
spring.datasource.password=password
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

User.java

```java
package com.example.demo.model;

import jakarta.persistence.*;
import lombok.*;

@Entity
@Data
@NoArgsConstructor
@AllArgsConstructor
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;
    private String name;
    private String email;
}
```

Java - Spring Boot + PostgreSQL (Spring Data JPA)

UserRepository.java

```java
package com.example.demo.repository;

import com.example.demo.model.User;
import org.springframework.data.jpa.repository.JpaRepository;

public interface UserRepository extends JpaRepository<User, Integer> {}
```

Java - Spring Boot + PostgreSQL (Spring Data JPA)

UserController.java

```java
package com.example.demo.controller;
import com.example.demo.model.User;
import com.example.demo.repository.UserRepository;
import org.springframework.web.bind.annotation.*;
import java.util.*;

@RestController
@RequestMapping("/users")
public class UserController {
    private final UserRepository repo;

    public UserController(UserRepository repo) {
        this.repo = repo;
    }

    @GetMapping
    public List<User> getAll() {
        return repo.findAll();
    }

    @GetMapping("/{id}")
    public User getById(@PathVariable Integer id) {
        return repo.findById(id).orElseThrow(() -> new RuntimeException("User not found"));
    }

    @PostMapping
    public User create(@RequestBody User user) {
        return repo.save(user);
    }

    @PutMapping("/{id}")
    public User update(@PathVariable Integer id, @RequestBody User newUser) {
        return repo.findById(id).map(u -> {
            u.setName(newUser.getName());
            u.setEmail(newUser.getEmail());
            return repo.save(u);
        }).orElseThrow(() -> new RuntimeException("User not found"));
    }

    @PatchMapping("/{id}")
    public User patch(@PathVariable Integer id, @RequestBody Map<String, Object> updates) {
        User user = repo.findById(id).orElseThrow(() -> new RuntimeException("User not found"));
        updates.forEach((key, value) -> {
            if (key.equals("name")) user.setName((String) value);
            if (key.equals("email")) user.setEmail((String) value);
        });
        return repo.save(user);
    }

    @DeleteMapping("/{id}")
    public Map<String, String> delete(@PathVariable Integer id) {
        repo.deleteById(id);
        return Map.of("message", "User deleted");
    }
}
```

Architektura warstwowa N-Layer

Architektura N-Layer (wielowarstwowa) to wzorzec organizacji aplikacji, który dzieli system na niezależne warstwy logiczne.

Warstwa prezentacji / kontrolerów
Warstwa modelu/serwisów/logiki biznesowej
Warstwa dostępu do danych

Cele:

- rozdzielić odpowiedzialności,
- zwiększyć modularność i testowalność,
- ułatwić utrzymanie i rozwój systemu.

Python - FastAPI + SQLAlchemy + Pydantic

```python
from sqlalchemy.orm import Session
from .. import models

class UserRepository:
    def get_all(self, db: Session):
        return db.query(models.User).all()

    def get_by_id(self, db: Session, id: int):
        return db.query(models.User).filter(models.User.id == id).first()

    def create(self, db: Session, user: models.User):
        db.add(user)
        db.commit()
        db.refresh(user)
        return user
```

```python
from ..repositories.user_repository import UserRepository
from .. import models

class UserService:
    def __init__(self):
        self.repo = UserRepository()

    def create_user(self, db, user_data):
        if not user_data.email.endswith("@example.com"):
            raise ValueError("Email musi pochodzić z domeny @example.com")
        new_user = models.User(**user_data.dict())
        return self.repo.create(db, new_user)
```

```python
from fastapi import APIRouter, Depends, HTTPException
from ..schemas import UserCreate, User
from ..database import get_db
from ..services.user_service import UserService

router = APIRouter(prefix="/users")
service = UserService()

@router.post("", response_model=User)
def create_user(user: UserCreate, db=Depends(get_db)):
    try:
        return service.create_user(db, user)
    except ValueError as e:
        raise HTTPException(400, detail=str(e))
```

C# - ASP.NET Core + EF Core

```
using UsersApi.Models;
using UsersApi.Data;
using Microsoft.EntityFrameworkCore;

public class UserRepository
{
    private readonly AppDbContext _db;
    public UserRepository(AppDbContext db) => _db = db;

    public async Task<List<User>> GetAllAsync() => await _db.Users.ToListAsync();
    public async Task<User?> GetByIdAsync(int id) => await _db.Users.FindAsync(id);
    public async Task<User> CreateAsync(User user)
    {
        _db.Users.Add(user);
        await _db.SaveChangesAsync();
        return user;
    }
}


public class UserService
{
    private readonly UserRepository _repo;
    public UserService(UserRepository repo) => _repo = repo;

    public async Task<User> CreateUserAsync(User user)
    {
        if (!user.Email.Contains("@")) throw new Exception("Niepoprawny email");
        return await _repo.CreateAsync(user);
    }
}
```

```
[ApiController]
[Route("[controller]")]
public class UserController : ControllerBase
{
    private readonly UserService _service;
    public UserController(UserService service) => _service = service;

    [HttpPost]
    public async Task<IActionResult> Create(User user)
    {
        try
        {
            var created = await _service.CreateUserAsync(user);
            return Ok(created);
        }
        catch (Exception ex)
        {
            return BadRequest(ex.Message);
        }
    }
}
```

JAVA - Spring Boot + JPA

```java
@Repository
public interface UserRepository extends JpaRepository<User, Integer> {}


@Service
public class UserService {
    private final UserRepository repo;

    public UserService(UserRepository repo) {
        this.repo = repo;
    }

    public User createUser(User user) {
        if (!user.getEmail().contains("@"))
            throw new IllegalArgumentException("Niepoprawny adres email");
        return repo.save(user);
    }
}
```

```java
@RestController
@RequestMapping("/users")
public class UserController {
    private final UserService service;

    public UserController(UserService service) {
        this.service = service;
    }

    @PostMapping
    public ResponseEntity<User> create(@RequestBody User user) {
        try {
            return ResponseEntity.ok(service.createUser(user));
        } catch (IllegalArgumentException e) {
            return ResponseEntity.badRequest().build();
        }
    }
}
```

BDD

Testowanie skupione na zachowaniu.

Scenariusze testowe są tworzone w języku naturalnym w formacie Gherkin (np. Given–When–Then)

- Discovery (Odkrywanie): Zespół omawia wymagania biznesowe i szczegółowe zachowania systemu.
- Formulation (Formułowanie): Wymagania są dokumentowane w postaci scenariuszy testowych w języku naturalnym, często w formacie Gherkin (np. w plikach .feature).
- Automation (Automatyzacja): Scenariusze są następnie mapowane na funkcje testowe w odpowiednim języku programowania, a testy są automatyzowane.

BDD

```
Feature: Zarządzanie użytkownikami
  Scenario: Utworzenie nowego użytkownika
    Given API działa
    When wysyłam żądanie POST na "/users" z:
      | Name | Email         |
      | Jan  | jan@test.com |
    Then odpowiedź ma status 200
    And odpowiedź zawiera użytkownika "Jan"
```

Python - FastAPI + Behave

```python
from behave import given, when, then
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)
response = None

@given('API jest uruchomione')
def step_api_running(context):
    assert client is not None

@when('wysyłam żądanie POST na "/users" z danymi:')
def step_send_post(context):
    global response
    for row in context.table:
        response = client.post("/users", json={"name": row['name'], "email": row['email']})

@then('odpowiedź powinna mieć status 200')
def step_check_status(context):
    assert response.status_code == 200

@then('odpowiedź powinna zawierać użytkownika "{name}"')
def step_check_user(context, name):
    data = response.json()
    assert data["name"] == name
```

bash: behave

C# - ASP.NET Core + SpecFlow

```
using System.Net.Http.Json;
using TechTalk.SpecFlow;
using Xunit;

[Binding]
public class UserSteps
{
    private readonly HttpClient _client;
    private HttpResponseMessage _response;
    private dynamic _data;

    public UserSteps()
    {
        _client = new HttpClient { BaseAddress = new Uri("http://localhost:5000") };
    }

    [Given(@"API działa")]
    public void GivenApiDziala() => Assert.NotNull(_client);

    [When(@"wysyłam żądanie POST na ""(.*)"" z:")]
    public async Task WhenSendPostRequest(string endpoint, Table table)
    {
        var row = table.Rows[0];
        var user = new { Name = row["Name"], Email = row["Email"] };
        _response = await _client.PostAsJsonAsync(endpoint, user);
        _data = await _response.Content.ReadFromJsonAsync<dynamic>();
    }

    [Then(@"odpowiedź ma status 200")]
    public void ThenStatusIs200() => Assert.Equal(200, (int)_response.StatusCode);

    [Then(@"odpowiedź zawiera użytkownika ""(.*)""")]
    public void ThenContainsUser(string name)
    {
        Assert.Equal(name, (string)_data.name.ToString());
    }
}
```

bash: dotnet test

Java - Spring Boot + Cucumber

```java
package com.example.demo.steps;

import io.cucumber.java.en.*;
import org.springframework.boot.test.context.SpringBootTest;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.web.client.TestRestTemplate;
import org.springframework.http.*;
import static org.junit.jupiter.api.Assertions.*;

@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
public class UserSteps {

    @Autowired
    private TestRestTemplate restTemplate;
    private ResponseEntity<String> response;

    @Given("API działa")
    public void apiDziala() {
        assertNotNull(restTemplate);
    }

    @When("wysyłam żądanie POST na {string} z danymi:")
    public void wysylamPost(String path, io.cucumber.datatable.DataTable table) {
        var row = table.asMaps().get(0);
        response = restTemplate.postForEntity(path, row, String.class);
    }

    @Then("odpowiedź ma status 200")
    public void checkStatus() {
        assertEquals(HttpStatus.OK, response.getStatusCode());
    }

    @Then("odpowiedź zawiera użytkownika {string}")
    public void containsUser(String name) {
        assertTrue(response.getBody().contains(name));
    }
}
```

```java
package com.example.demo;

import io.cucumber.junit.Cucumber;
import io.cucumber.junit.CucumberOptions;
import org.junit.runner.RunWith;

@RunWith(Cucumber.class)
@CucumberOptions(
    features = "src/test/resources/features",
    plugin = {"pretty"}
)
public class CucumberTest {}
```

bash: mvn test

Zadanie

Na ocenę 3.0:
Stwórz kontroler do zarządzania produktami, który będzie umożliwiał dodawanie, edycję, usuwanie oraz wyświetlanie produktów. Produkty muszą być przechowywane w bazie danych. Dostosuj aplikację do specyfikacji domenowej. Upewnij się, że implementacja spełnia wszystkie założenia przedstawione w specyfikacji.

+0.5 oceny:
Stwórz mechanizm blokady możliwości dodawania produktów o określonych nazwach. System powinien blokować dodanie oraz edycję produktu, którego nazwa zawiera frazy z listy zabronionych. Lista fraz zabronionych powinna być dynamiczna. Zaprojektuj nazewnictwo punktów końcowych zgodnie z dobrymi praktykami.

+0.5 oceny:
Podziel aplikację na warstwy zgodnie z architekturą N-Layer.

+0.5 oceny:
Dodaj w warstwie dostępu do danych mechanizm przechowywania historii zmian w produktach, np. śledzenie ilości dostępnych produktów, zmian w cenie czy innych właściwości. Każda zmiana powinna być zapisywana, aby można było przeglądać historię modyfikacji produktów. Zmiany danych muszą być przechowywane w bazie danych.

+0.5 oceny:
Implementacja testów integracyjnych, aby upewnić się, że aplikacja spełnia założenia biznesowe zgodnie ze specyfikacją. Muszą zostać pokryte wszystkie wymagania z specyfikacji domenowej.

-1.0 oceny:
Znaczące niezastosowanie się do zasad SOLID, DRY oraz KISS.

Zadanie - wymagane endpointy

Operacja	Metoda HTTP	Endpoint	Opis
Pobranie listy produktów	GET	/api/v1/products	Zwraca listę produktów z najważniejszymi informacjami (np. ID, nazwa, cena).
Pobranie szczegółów jednego produktu	GET	/api/v1/products/{id}	Zwraca pełne informacje o danym produkcie na podstawie jego ID.
Dodanie nowego produktu	POST	/api/v1/products	Tworzy nowy produkt na podstawie przesłanych danych.
Aktualizacja istniejącego produktu	PUT	/api/v1/products/{id}	Aktualizuje dane istniejącego produktu na podstawie jego ID.
Usunięcie produktu	DELETE	/api/v1/products/{id}	Usuwa produkt na podstawie jego ID.

Zadanie

Nazwa produktu:
• Musi być unikalna i składać się z co najmniej 3 znaków oraz maksymalnie 20 znaków.
• Może składać się jedynie z liter oraz cyfr.
• Nie mogą istnieć dwa produkty o tej samej nazwie.

Cena produktu musi spełniać minimalne i maksymalne wartości zależne od kategorii danego produktu, np.:
• Elektronika: cena min. 50 PLN, max. 50 000 PLN
• Książki: cena min. 5 PLN, max. 500 PLN
• Odzież: cena min. 10 PLN, max. 5 000 PLN

Ilość dostępnych produktów:

• Musi być liczbą całkowitą i wynosić co najmniej 0 (nie może być ujemna).

Obsługa wyjątków:

• W przypadku próby dodania lub edycji produktu niezgodnego z powyższymi zasadami, aplikacja powinna wyświetlić odpowiednie komunikaty błędów, wskazując pole, które wymaga poprawy.

Zadanie

Projekt realizowany w dowolnym języku spełniającym wymagania projektu.

- Kod powinien być czytelny i zgodny z zasadami clean code (SOLID, DRY, KISS).
- Uwzględnienie obsługi błędów i walidacji danych wejściowych,
- Informacje o produktach muszą być przechowywane w bazie danych,
- Wykorzystanie metod HTTP powinno być zgodne ze standardem,
- Nazewnictwo punktów końcowych zgodnie z konwencją,
- Komunikacja testów z API poprzez HTTP (np. Refit)